

## Problem Session 4: Memory Class and API in C Programming

|                       |             |
|-----------------------|-------------|
| Kiatissak Markmeeshap | 67070501005 |
| Tithinan Sobking      | 67070501018 |
| Worawut Sereethai     | 67070501040 |
| Chanya Poolketkij     | 67070501058 |

All tests were executed on a single Ubuntu environment (WSL2 on Windows 11), using exactly the two commands specified — `gcc -o exe src.c` and `gcc -no-pie -o exe src.c` — with no additional flags, so that no compiler option can account for the addresses reported below. Items 1 and 2 were evaluated under both configurations across three runs each; item 3 was executed without `-no-pie`.

### 1 Static Storage Class

#### 1.1 The original program

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         static int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }
```

Listing 1: Source: src/1\_static.c

```

--- 1_static.c, default PIE, run 1 ---
The value of y is 6
The address of y is 0x58cb201ad010

The value of y is 7
The address of y is 0x58cb201ad010

The value of y is 8
The address of y is 0x58cb201ad010

--- 1_static.c, default PIE, run 2 ---
The value of y is 6
The address of y is 0x5a3ca3532010

The value of y is 7
The address of y is 0x5a3ca3532010

The value of y is 8
The address of y is 0x5a3ca3532010

--- 1_static.c, default PIE, run 3 ---
The value of y is 6
The address of y is 0x5fc5b0b79010

The value of y is 7
The address of y is 0x5fc5b0b79010

The value of y is 8
The address of y is 0x5fc5b0b79010

```

**Listing 2:** All three runs of the original program

Declaring `y` as `static` allocates it in the initialised data segment rather than the stack, giving it static storage duration. The initialisation (`= 5`) occurs only once, at program startup. Subsequent iterations execute only `y++`, producing the values 6, 7 and 8 at an invariant address. (auto on `x` is the default for local variables and has no effect.)

## 1.2 With static removed

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }

```

**Listing 3:** Source: `src/1_static_no_static.c`

```

--- 1_static_no_static.c, default PIE, run 1 ---
The value of y is 6
The address of y is 0x7fff9688e0f0

The value of y is 6
The address of y is 0x7fff9688e0f0

The value of y is 6
The address of y is 0x7fff9688e0f0

--- 1_static_no_static.c, default PIE, run 2 ---
The value of y is 6
The address of y is 0x7ffee90a8000

The value of y is 6
The address of y is 0x7ffee90a8000

The value of y is 6
The address of y is 0x7ffee90a8000

--- 1_static_no_static.c, default PIE, run 3 ---
The value of y is 6
The address of y is 0x7ffc8f16a3e0

The value of y is 6
The address of y is 0x7ffc8f16a3e0

The value of y is 6
The address of y is 0x7ffc8f16a3e0

```

**Listing 4:** All three runs after removing `static`

Without `static`, `y` becomes an automatic variable allocated on the stack. It is reinitialised to 5 upon entering each loop iteration and deallocated upon exiting, yielding the value 6 on every pass. The address falls within the high stack range (`0x7ff...`); it remains identical across iterations due to stack slot reuse rather than state persistence.

### 1.3 Compiled with `-no-pie`

```

--- 1_static.c, -no-pie, run 1 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

--- 1_static.c, -no-pie, run 2 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

--- 1_static.c, -no-pie, run 3 ---
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

```

**Listing 5:** The original program rebuilt with `-no-pie`

```

--- 1_static_no_static.c, -no-pie, run 1 ---
The value of y is 6
The address of y is 0x7fffd2a525e0

The value of y is 6
The address of y is 0x7fffd2a525e0

The value of y is 6
The address of y is 0x7fffd2a525e0

--- 1_static_no_static.c, -no-pie, run 2 ---
The value of y is 6
The address of y is 0x7ffe40c85ce0

The value of y is 6
The address of y is 0x7ffe40c85ce0

The value of y is 6
The address of y is 0x7ffe40c85ce0

--- 1_static_no_static.c, -no-pie, run 3 ---
The value of y is 6
The address of y is 0x7ffcca719fb0

The value of y is 6
The address of y is 0x7ffcca719fb0

The value of y is 6
The address of y is 0x7ffcca719fb0

```

**Listing 6:** The static-less program with `-no-pie`

| Build                  | Run 1          | Run 2          | Run 3          |
|------------------------|----------------|----------------|----------------|
| static, default PIE    | 0x58cb201ad010 | 0x5a3ca3532010 | 0x5fc5b0b79010 |
| static, -no-pie        | 0x404018       | 0x404018       | 0x404018       |
| no static, default PIE | 0x7fff9688e0f0 | 0x7ffee90a8000 | 0x7ffc8f16a3e0 |
| no static, -no-pie     | 0x7fffd2a525e0 | 0x7ffe40c85ce0 | 0x7ffcca719fb0 |

Table 1: Address of `y` in each of the three runs

The two builds do produce different results, as the table shows.

By default `gcc` generates a position-independent executable (PIE) whose base address is randomised by ASLR on each execution, causing the address of static `y` to change across runs. Compiling with `-no-pie` fixes the program’s load address, pinning `y` to `0x404018`. The values (6, 7, 8) remain unaffected, as storage duration is governed by language semantics, not by binary loading.

Notably, the non-static `y` moves across runs even under `-no-pie`. Because stack layout is randomised independently by the kernel, disabling PIE fixes only the text and data segments, not the stack.

## 2 Extern Storage Class

### 2.1 The original program

```

1 #include <stdio.h>
2
3 int x = 20;
4
5 static void display(void)
6 {
7     extern int x;
8     printf("Display value of x: %d\n", x);
9     printf("Address of x (in display function): %p\n\n", (void *)&x);
10 }
11
12 int main(void)
13 {
14     extern int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }

```

Listing 7: Source: `src/2_extern.c`

```

--- 2_extern.c, default PIE, run 1 ---
Print value of x: 20
Address of x (in main function): 0x5e847872a010

Display value of x: 20
Address of x (in display function): 0x5e847872a010

--- 2_extern.c, default PIE, run 2 ---
Print value of x: 20
Address of x (in main function): 0x55bd30571010

Display value of x: 20
Address of x (in display function): 0x55bd30571010

--- 2_extern.c, default PIE, run 3 ---
Print value of x: 20
Address of x (in main function): 0x60475102d010

Display value of x: 20
Address of x (in display function): 0x60475102d010

```

**Listing 8:** All three runs of the original program

`extern int x;` declares `x` without defining it, instructing the compiler to reference the existing file-scope definition (`int x = 20;`). Both functions therefore reference the same object in the data segment, printing identical values (20) and addresses. While redundant here within a single file, `extern` makes shared linkage explicit and is how one translation unit reaches a variable defined in another.

## 2.2 With `extern` removed from `main()`

```

1  #include <stdio.h>
2
3  int x = 20;
4
5  static void display(void)
6  {
7      extern int x;
8      printf("Display value of x: %d\n", x);
9      printf("Address of x (in display function): %p\n\n", (void *)&x);
10 }
11
12 int main(void)
13 {
14     int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }

```

**Listing 9:** Source: `src/2_extern_no_extern.c`

```

--- 2_extern_no_extern.c, default PIE, run 1 ---
Print value of x: 0
Address of x (in main function): 0x7ffcd6c9e714

Display value of x: 20
Address of x (in display function): 0x56f9dc7b3010

--- 2_extern_no_extern.c, default PIE, run 2 ---
Print value of x: 0
Address of x (in main function): 0x7ffdc5b931d4

Display value of x: 20
Address of x (in display function): 0x586cd516f010

--- 2_extern_no_extern.c, default PIE, run 3 ---
Print value of x: 0
Address of x (in main function): 0x7ffc34bb6704

Display value of x: 20
Address of x (in display function): 0x5bd01b79b010

```

**Listing 10:** All three runs after removing `extern`

Removing `extern` turns `int x`; inside `main()` into a local definition on the stack, shadowing the global variable. Because this local variable is uninitialised, reading it invokes undefined behaviour — as the compiler warns: ‘`x`’ is used uninitialized — outputting arbitrary stack content and a stack-range address. Meanwhile `display()` is unchanged and continues to access the global variable in the data segment. The two addresses printed within a single run lie in entirely different regions of memory, which is the clearest evidence that they are two distinct variables.

## 2.3 Compiled with `-no-pie`

```

--- 2_extern.c, -no-pie, run 1 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

--- 2_extern.c, -no-pie, run 2 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

--- 2_extern.c, -no-pie, run 3 ---
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

```

**Listing 11:** The original program rebuilt with `-no-pie`

```

--- 2_extern_no_extern.c, -no-pie, run 1 ---
Print value of x: 0
Address of x (in main function): 0x7fff9a126b64

Display value of x: 20
Address of x (in display function): 0x404020

--- 2_extern_no_extern.c, -no-pie, run 2 ---
Print value of x: 0
Address of x (in main function): 0x7ffe59e39194

Display value of x: 20
Address of x (in display function): 0x404020

--- 2_extern_no_extern.c, -no-pie, run 3 ---
Print value of x: 0
Address of x (in main function): 0x7ffebaabcea4

Display value of x: 20
Address of x (in display function): 0x404020

```

**Listing 12:** The extern-less program with `-no-pie`

| Build                                 | Run 1          | Run 2          | Run 3          |
|---------------------------------------|----------------|----------------|----------------|
| global x, default PIE                 | 0x5e847872a010 | 0x55bd30571010 | 0x60475102d010 |
| global x, <code>-no-pie</code>        | 0x404018       | 0x404018       | 0x404018       |
| local x in main, default PIE          | 0x7ffcd6c9e714 | 0x7ffdc5b931d4 | 0x7ffc34bb6704 |
| local x in main, <code>-no-pie</code> | 0x7fff9a126b64 | 0x7ffe59e39194 | 0x7ffebaabcea4 |

Table 2: Address of x in each of the three runs

As observed in item 1, compiling with `-no-pie` pins the global `x` to a fixed address (0x404018), whereas the default PIE build subjects it to ASLR. The local `x` in `main()` resides on the stack, where both the address and the uninitialised value vary on every run regardless of the PIE setting.

## 3 Memory Allocation

### 3.1 The original program

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      int *a;
7      float *b;
8      int c[10];
9
10     printf("\nAddress of Pointer\n");
11     printf(">>>p\n", (void *)&a);
12     printf(">>>p\n", (void *)&b);
13
14     printf("\nEffective Address\n");
15     printf(">>>p\n", (void *)a);
16     printf(">>>p\n", (void *)b);
17
18     a = malloc(10 * sizeof(int));
19     printf("\nAfter malloc Pointer a\n");
20     printf(">>>p\n", (void *)a);
21     printf(">>>p\n", (void *)&a[0]);
22     printf(">>>p\n", (void *)&a[9]);

```

```

23     printf("\nArray c\n");
24     printf(">>%p\n", (void *)c);
25     printf(">>%p\n", (void *)&c[0]);
26     printf(">>%p\n", (void *)&c[9]);
27
28
29     a = realloc(a, 1000 * sizeof(int));
30     printf("\nAfter realloc Pointer a\n");
31     printf(">>%p\n", (void *)a);
32     printf(">>%p\n", (void *)&a[0]);
33     printf(">>%p\n", (void *)&a[9]);
34     printf(">>%p\n", (void *)&a[999]);
35
36     free(a);
37     return 0;
38 }

```

**Listing 13:** Source: `src/3_memory.c`

```

--- 3_memory.c, default PIE, run 1 ---

Address of Pointer
>>0x7ffed482f730
>>0x7ffed482f738

Effective Address
>>0x1800000017
>>(nil)

After malloc Pointer a
>>0x558fb7513020
>>0x558fb7513020
>>0x558fb7513044

Array c
>>0x7ffed482f740
>>0x7ffed482f740
>>0x7ffed482f764

After realloc Pointer a
>>0x558fb7513020
>>0x558fb7513020
>>0x558fb7513044
>>0x558fb7513fbc

```

**Listing 14:** The program as given

**Address of Pointer** (`&a`, `&b`) reflects the local pointer variables on the stack, separated by 8 bytes, the pointer size on this 64-bit architecture.

**Effective Address** prints the values held in `a` and `b` before any allocation. Reading them is undefined behaviour: they hold whatever the stack happened to contain. In this run one evaluated to a nonsensical address and the other to `(nil)`, which is precisely the point — `(nil)` is not guaranteed, and an uninitialised pointer is not a null pointer.

**After malloc** shows `a` pointing to dynamic memory on the heap, far from the stack. `a` and `&a[0]` reference the start of the buffer, while `&a[9]` is offset by 36 bytes ( $9 \times 4$  bytes per `int`).

**Array c** is allocated on the stack alongside the other automatic variables, requiring no dynamic allocation and permitting no explicit deallocation; its lifetime ends with the function.

**After realloc** preserves `a`'s base address, because no allocation was made immediately after it and the block could be extended in place, up to `&a[999]` at 3996 bytes past the base.

## 3.2 With the allocation of `b` enabled

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)

```

```

5 {
6     int *a;
7     float *b;
8     int c[10];
9
10    printf("\nAddress of Pointer\n");
11    printf(">>>p\n", (void *)&a);
12    printf(">>>p\n", (void *)&b);
13
14    printf("\nEffective Address\n");
15    printf(">>>p\n", (void *)a);
16    printf(">>>p\n", (void *)b);
17
18    a = malloc(10 * sizeof(int));
19    printf("\nAfter malloc Pointer a\n");
20    printf(">>>p\n", (void *)a);
21    printf(">>>p\n", (void *)&a[0]);
22    printf(">>>p\n", (void *)&a[9]);
23
24    printf("\nArray c\n");
25    printf(">>>p\n", (void *)c);
26    printf(">>>p\n", (void *)&c[0]);
27    printf(">>>p\n", (void *)&c[9]);
28
29    b = malloc(10 * sizeof(float));
30    printf("\nAfter malloc Pointer b\n");
31    printf(">>>p\n", (void *)b);
32    printf(">>>p\n", (void *)&b[0]);
33    printf(">>>p\n", (void *)&b[9]);
34
35    a = realloc(a, 1000 * sizeof(int));
36    printf("\nAfter realloc Pointer a\n");
37    printf(">>>p\n", (void *)a);
38    printf(">>>p\n", (void *)&a[0]);
39    printf(">>>p\n", (void *)&a[9]);
40    printf(">>>p\n", (void *)&a[999]);
41
42    free(a);
43    free(b);
44    return 0;
45 }

```

Listing 15: Source: src/3\_memory\_with\_b.c

```
--- 3_memory_with_b.c, default PIE, run 1 ---
```

```
Address of Pointer
```

```
>>0x7fffe767fea0
```

```
>>0x7fffe767fea8
```

```
Effective Address
```

```
>>0x1800000017
```

```
>>(nil)
```

```
After malloc Pointer a
```

```
>>0x5bf54a021020
```

```
>>0x5bf54a021020
```

```
>>0x5bf54a021044
```

```
Array c
```

```
>>0x7fffe767feb0
```

```
>>0x7fffe767feb0
```

```
>>0x7fffe767fed4
```

```
After malloc Pointer b
```

```
>>0x5bf54a021050
```

```
>>0x5bf54a021050
```

```
>>0x5bf54a021074
```

```
After realloc Pointer a
```

```
>>0x5bf54a021080
```

```
>>0x5bf54a021080
```

```
>>0x5bf54a0210a4
```

```
>>0x5bf54a02201c
```

**Listing 16:** The same program with **b** allocated

| Build     | After malloc   | After realloc  |
|-----------|----------------|----------------|
| without b | 0x558fb7513020 | 0x558fb7513020 |
| with b    | 0x5bf54a021020 | 0x5bf54a021080 |

Table 3: Where **a** lives before and after `realloc()`

The result does differ from the previous run: with **b** allocated, **a** moves.

Allocating **b** immediately after **a** prevents in-place expansion during `realloc(a, 1000 * sizeof(int))`. Because **b**'s chunk obstructs contiguous growth, the allocator relocates **a** to a sufficiently large free block, copies the existing contents, releases the old block and returns a new address.

Whether a block moves is therefore not a property of the program but of what else has been allocated around it: the `realloc` call is identical in both versions. This is why its return value must always be reassigned to the pointer, and why any other pointer into the old block is invalid afterwards.